

Post-agility: What follows a decade of agility?

Richard Baskerville^a, Jan Pries-Heje^b, Sabine Madsen^{b,*}

^a Georgia State University, University Plaza, Atlanta, GA 30302, USA

^b Department of Communication, Business and IT, Roskilde University, Universitetsvej 1, DK-4000 Roskilde, Denmark

ARTICLE INFO

Article history:

Available online 20 November 2010

Keywords:

Agile information systems development
Agility
Grounded theory

ABSTRACT

Context: Agile information systems development (ISD) has received much attention from both the practitioner and researcher community over the last 10–15 years. However, it is still unclear what precisely constitutes agile ISD.

Objective: Based on four empirical studies conducted over a 10-year time period from 1999 to 2008 the objective of this paper is to show how the meaning and practice of agile ISD has evolved over time and on this basis to speculate about what comes next.

Method: Four phases of research has been conducted, using a grounded theory approach. For each research phase qualitative interviews were held in American and/or Danish companies and a grounded theory was inductively discovered by careful data analysis. Subsequently, the four unique theories have been analyzed for common themes, and a global theory was identified across the empirical data.

Results: In 1999 companies were developing software at high-speed in a desperate rush to be first-to-market. In 2001 a new high-speed/quick results development process had become established practice. In 2003 changes in the market created the need for a more balanced view on speed and quality, and in 2008 companies were successfully combining agile and plan-driven approaches to achieve the benefits of both. The studies reveal a two-stage pattern in which dramatic changes in the market causes disruption of established practices and process adaptations followed by consolidation of lessons learnt into a once again stable software development process.

Conclusion: The cyclical history of punctuated process evolution makes it possible to distinguish pre-agility from current practices (agility), and on this basis, to speculate about post-agility: a possible next cycle of software process evolution concerned with proactively pursuing the dual goal of agility and alignment through a diversity of means.

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Agile information systems development (ISD) has received much attention from both the practitioner and researcher community over the last 10–15 years. First as a novelty and later as a development approach that has become widely used in practice [1]. Agile ISD is often referred to as high-speed development [2]; an approach for dealing with change [3]; and as characterized by intensively iterative processes [4]. Moreover, in the agile manifesto from 2001 a number of values and principles for conducting agile ISD were specified (agilemanifesto.org), and numerous agile methods emerged. However, to date it is still unclear what precisely constitutes agile ISD.

Concepts provide the language for understanding and for being understood as well as for making and comparing empirical observations [5]. Without a clear definition it is difficult to determine if a

particular ISD method or method component actually contributes to the development team's ability to achieve agility; to compare agile methods; or to evaluate the suitability of agile and plan-driven methods for different types of development projects and contexts [3]. More conceptual studies have therefore been called for [6], among other reasons, because "...conceptual studies are required to develop much-needed instruments to assess the agility of ISD processes and teams, whether co-located or distributed" [2, p. 323].

Attempts to clarify the concept of agility in an ISD context advocates a historical perspective [4] and 'going back to the roots' [3]. Based on a review of manufacturing and management literature Conboy [3] applies an iterative approach to define agility and to propose an instrument for assessing the agility of a particular practice, regardless of whether the practice is referred to as agile, plan-driven, or in-house. The definition and taxonomy emphasizes *speed*, *change*, *learning*, and *customer value* as the constitutive elements of agility. The definition of an ISD method's ability to support agility is as follows: "the continual readiness of an ISD method to rapidly or inherently create change, proactively or reactively

* Corresponding author. Tel.: +45 46743790.

E-mail addresses: Baskerville@acm.org (R. Baskerville), Janph@ruc.dk (J. Pries-Heje), Sabinem@ruc.dk (S. Madsen).

embrace change, and learn from change while contributing to perceived customer value (economy, quality, and simplicity), through its collective components and relationships with its environment” [3, p. 340].

In this paper we provide a historical overview of the evolution of agile ISD with regard to speed, change, learning, and customer value as a result of changes in the business context. Based on empirical data, we show that agile ISD has emerged in a series of iterations that each have further developed, and defined the meaning and practice of agile ISD. Thus, our aim is to show *how agile ISD – as a practical phenomenon – has evolved until now* and on this basis, to speculate about *what comes next*.

Our empirical foundation is a 10-year series of studies of ISD that reveals how agile practices have evolved and changed. We compare and contrast the practices of high-speed and agile ISD at four different points in time: When the internet was booming in 1999; during the peak of the “dot.com” boom in 2000–2001; just after this economy collapsed in 2002–2003; and most recently in 2008 when agile ISD had become widely used in practice. For simplicity, the four studies and points in time are here after referred to as study one from 1999, study two from 2001, study three from 2003, and study four from 2008.

An earlier version of this research has been published as a book chapter [7]. The book chapter presents the four empirical studies and their findings and identifies the two-stage pattern of market- and process-driven change. However, in the book chapter, the empirical results were only tentatively positioned and related to prior research. In this paper we therefore develop the research further by drawing on a method taxonomy [3] to discuss our findings at a more conceptual level. We also introduce the concepts of *pre-agility*, *agility*, and *post-agility* to link the empirical findings to the past, present, and future of agile ISD, thereby placing the research in its historical context.

The paper contributes to existing research with the following insights. First, each empirical study delineates the development practices that were used at the time of the study. Many agile practices have roots in much older methods and concepts [3]. Therefore, we are less concerned with whether the reported practices are “new” to IS research as such. Instead we focus on what the involved practitioners perceived to be new and changing about their practice at a given point in time. Also, for each study period we highlight the agile aspects that were the most prevalent. Second, the four studies collectively facilitate an understanding of how agile ISD in practice has changed over time – from high-speed development for the Internet conducted in an ad hoc manner without a common label to a development approach with its own name, values, principles, and practices. Thus, even though agile ISD might still (from a research and theory building point of view) be conceptually weak, our findings shows that in practice the concept is now distinct and well-defined enough for practitioners to explicitly explain how they use and combine the agile approach with plan-driven ways of working to harvest the benefits of both. Third, explanations for the evolution of agile ISD into a distinct phenomenon are provided through delineation of how local software processes have interacted with global changes in the software development context over the 10-year time period. This allows for an understanding of the evolution of agile processes as historical cycles that are both punctuated and driven forward by radical change in the market. Each episode defines the solutions to the new contextual challenges by emphasizing some and defocusing other aspects, thereby in turn shaping the problems that have to be solved in the next cycle.

We have organized the remainder of the paper as follows. First we describe our research methodology and use of grounded theory. We then summarize the individual story line that proceeds from each of the four study periods and highlight the agile aspects (i.e.

speed, change, learning, customer value) that were prevalent in each study. Lastly we conclude with a discussion of the overall story line that helps explain how and why agile ISD has evolved over the 10-year time span and on this basis, we suggest the notion of post-agility.

2. Research methodology

We have undertaken four phases of research, using grounded theory (GT) as our research methodology. GT is a qualitative research methodology that takes its name from the practice of discovering theory that is grounded in data. This research methodology does not begin with a theory, and then seek proof; rather it starts with an area of study and allows the relevant theory to emerge from that area [8]. The research outcome is grounded theories that are inductively discovered by careful collection and analysis of qualitative, empirical data. Use of GT in IT research is exemplified by a landmark paper by Orlikowski [9] on CASE tools and organizational change, as well as explorations on software requirements [10,11]. GT is best used in research where one has relatively “uncharted land”, which for example was the case with high-speed development for the Internet.

When the research presented in this paper was initiated agile ISD had not yet been coined as a term and phenomenon of its own. Yet it was clear that something was happening due to the rapid adoption of the Internet. We compared the adoption rate to the explosion of a bomb and referred to development for the Internet as “racing the e-bomb” [12]. In 1999 and 2001 our research questions therefore revolved around the concept of ‘Internet speed’: What does it mean? Is there something one could distinguish as “Internet speed development”? How is it different from or similar to traditional development? In 2003, we continued to ask about Internet speed, but were more focused on what had changed from the boom to the bust. Five years later agile development had become an established term that captured and moved beyond the early manifestations of agility that we noted already in 1999. Thus, in 2008, agile methods, and especially Scrum [13] and eXtreme Programming (XP) [14–16], had become popular in practice. In the literature, however, the ideal settings for the use of agile versus plan-driven methods were much discussed (see e.g. [17–19]), while in practice agile methods seemed to be successfully used, also outside the initially recommended application area. Our research interest therefore centered on the question of how agile development, and more specifically the agile method, Scrum, was used in projects at or well beyond the edge of its original ‘sweet spot’ [19] and why this seemed to work.

For all four phases of research, we have collected our data via semi-structured interviews. The interview guide was structured around the following topics:

1. The firm, and its’ products and services.
2. The interviewee.
3. Projects in the organization – from start to end.
4. Development model used?
5. Internet time/agile development – What does it mean to you?
6. The development process itself.
7. Talent, training, learning, and knowledge.
8. Transfer of knowledge.
9. The biggest problem/greatest challenge?

Each interview lasted approx. 1–1½ h, relevant documents were collected, and observation notes were recorded (e.g. about the use of open- or closed space offices; and the general impression of the pace, atmosphere, and ‘tone’ of the work place).

For data analysis we have applied the three coding procedures of GT [8] called open, axial and selective coding.

The goal of open coding is to reveal the core ideas found in the data. Open coding involves two tasks. The first task is labeling phenomena. This task involves decomposing observations into discrete incidents or ideas. Each discrete incident or idea receives a name or label that represents the phenomenon. These names represent a concept inherent in the observation. The second task is discovering categories. Categorizing is the process of finding related phenomena or common concepts and themes in the accumulated data in order to group them under joint headings, thus identifying categories and sub-categories of data.

The purpose of axial coding is to develop a deeper understanding of how the identified categories are related. Axial coding also involves two tasks. The first task connects categories in terms of a sequence of relationships. For example, a causal condition or a consequence can connect two categories, or a category and a sub-category. The second task turns back to the data for validation of the relationships. This return gives rise to the discovery and specification of the differences and similarities among and within the categories.

Selective coding involves the integration of the categories that have been developed to form the initial theoretical framework. First, a story line is generated or made explicit. A story is simply a descriptive narrative about the central phenomenon of study; the story line is the conceptualization of this story (abstracting). The story line becomes the core category, which is related to all the categories found during axial coding, thereby validating these relationships, and elaborating the categories into a theoretical expression that explains the phenomena observed.

2.1. Study one: Interview study in Denmark

The first phase of our research aimed at exploring the influence of working on Internet time [20]. One could say that we were testing the hypothesis that working on Internet time would have to cause some changes in the way software development work was organized. But beyond this no hypotheses were pre-formulated and tested.

We interviewed in three Danish companies. Two of the companies were new to the authors and the third was a company we had visited over a period of time for a longitudinal study. The main facts about the three companies are given in Table 1, and further details can be found in Baskerville and Pries-Heje [12].

2.2. Study two: Interview study in USA

The second phase of our research involved ten detailed case studies of Internet software development companies in two major

U.S. metropolitan areas. The firms ranged in size from 10 employees to more than 300,000 employees and covered different industries in the private and public sectors including: financial services, insurance, business and consulting services, courier services, travel, media, utilities, and government services. Some of the firms were Internet start-ups while others were “brick and mortar” companies with newly established Internet development units. A brief description of each firm is provided in Table 2, and further details are available in Baskerville et al. [21] and Pries-Heje et al [22].

The objective of study two was to understand whether software development for the Internet differs from traditional software development. This phase identified the practices used for Internet software development and explored the role of quality in fast-cycle development environments [23].

2.3. Study three: A follow-up study

Another round of interviews in the same companies as in phase 2 was conducted 2 years later. Only five of the original ten companies (from 2001) remained in business or were available to participate in the study (see the 2003 entries in Table 2).

2.4. Study four: Scrum interview study in Denmark

The fourth round of interviews was conducted for the purpose of exploring how Scrum was used in projects characterized by larger and geographically distributed teams concerned with the development of business and life critical software. Three Danish companies were selected as relevant sites for data collection as their IT projects exhibited these characteristics (see Table 3). The case companies had from one year to two and a half years of experience with the use of Scrum, with SuperSystem being the most experienced.

The results of the four phases of research are presented below in the form of four grounded theories. The theories cover several levels of analysis, namely the market, the portfolio, the project, and the team level. However, many of our respondents were operating at the project and team level (project managers and developers). We have therefore been able to collect more detailed data, conduct more thorough analyses, and develop more robust theories about these two levels.

3. The four study periods and their storylines

In this section we describe the evolution from high-speed development for the Internet to the emergence of agile ISD as a fully-fledged, practical phenomenon. We present the storyline and properties of the development method for each of the four

Table 1
Facts about the three companies (study one).

Name (Pseudonym)	What offered? When founded? Which size?	Number of people interviewed and their organizational roles
NewWays	Develops custom-tailored Internet products for major customers internationally Founded in the mid 1990s 50 employees when interviewed	Four people interviewed: One project manager, a development manager and two developers.
ProfWeb	Develops custom-tailored Internet and Intranet products interfacing with large existing databases Founded in the early 1990s 40 employees when interviewed	Two people interviewed: A development manager and a developer.
AlfaWeb	A general web-based product sold on the market as a standard product for e-commerce Founded in the late 1990s 12 employees when interviewed	Two people interviewed: The CEO and a development manager.

Table 2

Facts about the ten companies (study two and three).

Name (Pseudonym)	What offered? When founded? Which size?	Number of people interviewed in each round and their organizational roles
Calliope	Offers forecasting tools for energy and communications industry Founded in the mid 1990s 20 employees when interviewed	2001: Three people interviewed: VP Operations, Project Manager, Software Developer 2003: Not interviewed
Clio	Low-price health care and utilities for groups of customers Founded in the late 1990s 35 employees when interviewed	2001: Six people interviewed: President & CEO, VP Technology Operations, Director of Marketing Research, Chief Information Officer, two developers 2003: Not interviewed
Deca	Develops and markets a platform of E-business software modules that allow users more control when doing business online Founded in the late 1990s Approx. 10 employees when interviewed	2001: Not interviewed 2003: Four people interviewed: CEO, developer, QA specialist and marketing manager
Erato	Offers to help Brick & Mortar companies get online Founded in the late 1990s 55 employees when interviewed	2001: Four people interviewed: Director, Chief Financial Officer, Chief Operations Officer, and developer 2003: Not interviewed
Euterpe	Film and Television Industry. Offers high-tech tools online Founded in the mid 1990s 80 employees when interviewed	2001: Four people interviewed: Project managers, marketing specialists, senior web developers 2003: Not interviewed
Melpomene	Carries out personnel administration for other companies online Founded in the mid 1990s More than 100 employees when interviewed	2001: Seven people interviewed: Project managers, process improvers, architects, user interface designers, web developers 2003: 6 of 7 people interviewed. Process improvement person had left company
Polyhymnia	Offers online services for transport and tourist industry Founded in the early 1990s More than 1000 employees when interviewed	2001: Six people interviewed: Senior managers, Project managers, QA manager, lead developers, web developer 2003: Seven people interviewed: Same distribution of roles as in 2001
Terpsichore	Offers industrial insurance online Founded in the 1930s More than 10,000 employees when interviewed	2001: Three people interviewed: Human Resources Manager, Internet site manager and Internet site developer 2003: Not interviewed
Thalia	Online service for transport and logistics industry Founded in the 1930s More than 100,000 employees when interviewed	2001: Six people interviewed: CIO, Senior manager, project managers, architects, senior developers, web developers 2003: Three of the six people interviewed: CIO, senior and project manager
Urania	Business-to-business communication Founded in the 1980s More than 100,000 employees when interviewed	2001: Six people interviewed: Senior manager, Project managers, quality assurance manager, QA specialist, Web developers 2003: Six people interviewed. Same roles. But only three were the same people

Table 3

Facts about the three companies (study four).

Name (Pseudonym)	What offered? When founded? Which size?	Number of people interviewed in each round and their organizational roles
GlobeRiver	Develops engineering products with built-in intelligence (software) Founded in 1940s 500 employees in R&D function world-wide when interviewed; of this 25 in a company-owned development house in India (Developers and Scrum masters)	Three people interviewed: a Danish Scrum master, a Danish Facilitator, and an Indian Scrum master.
SuperSystem	Develops software for the military, the banking industry, hospitals, etc. Founded in 1980s Approx. 400 employees when interviewed	Four people interviewed: a Lead developer, a Scrum master, the manager of the internal software process improvement (SPI) department, and the person officially in charge of implementing Scrum in the company.
DareYou	An off- and online gaming company; works with several suppliers located in different places and countries to develop the online games Founded in 1940s Approx. 250 employees when interviewed	Two people interviewed: The Project manager and the Product owner.

study periods. Moreover, for each study, we discuss which agile aspects that were the most prevailing at the time. To assess the

method's contribution to agility at the four points in time, we draw on Conboy's [3] definition and taxonomy.

- Be continually ready, i.e. it must take minimal time and cost to prepare the component for use.
- Contribute to one or more of the following: rapid or inherent creation of change, proaction in advance of change, reaction to change, learning from change.
- Contribute to one or more of the following, and must not detract from any: perceived economy, quality, or simplicity.

In the first study we noted ten properties of a new methodology for “e” development [12]. Each of these properties is briefly described below, along with examples of how these properties are manifested in the cases. We also describe the chain of causal links that we discovered among these properties, which helps explain why this particular set of properties has come to characterize Internet time development (an early manifestation of agility). These properties and the causal chain are depicted graphically in Fig. 1.

We found time pressure to be a condition permeating software development in the three companies we studied. First-to-market is the central, defining high-priority goal of Internet time development. Minimizing time-to-market from concept to customer is an all-consuming activity and achievement of this goal drives almost all other elements of the methodology. This goal is not new in business [24] nor in software development [25,26]. However, the degree to which it influenced systems development had not yet been recognized when we conducted this study.

An inability to pre-define system requirements is the central, defining constraint of Internet time development. The requirements specification has traditionally been the heart of systems development. However, Internet time development accepts a starting point in which the requirements are permitted to persist in near or full ambiguity. For example a project manager at NewWays said, "Often a project starts without a requirements specification. . . .companies come to us and say: We believe there is a treasure buried in the World Wide Web. . . . we want something new".

The idea of using prototypes seems to be widespread and permeating both early and late work in development projects. For example ProfWeb describes their use of prototypes as being part of their core competence. The R&D manager said: "We live from being technologically in front of our competitors, and from being able to visualise more far-reaching and wide-ranging solutions for our customers than our competitors are able to".

The vague requirements are not just something we see in the beginning of a project. In fact it continues throughout the development process. One consequence is what we have named a “release orientation”. Software systems are produced in a series of ever more refined and extensive versions of the product; and each release contains bug-fixes and new features. These maturing product cycles characterize Internet software development in which competition demands significant product and feature changes every few months [25, p. 299]. This release orientation helps relieve some of the time pressure because if a feature does not make it for the contemporary release, it can simply be postponed to the following release, which is never very far behind.

The release orientation demands a fast cycle time that is impossible to meet in a serial process. Parallel development processes therefore flourish, meaning that a number of activities take place at the same time. Products and releases therefore have to be designed and coordinated for parallel development, another aspect common to Internet software development [25, p. 14]. For example, NewWays projects typically have a duration of 2–3 months. A sequential, waterfall-like model is seen as much too slow. Instead NewWays have several parallel development processes running at the same time.

To make parallel development possible, it is necessary to have some means for dividing the work. In all three cases we found that this has led to the use of a fixed three-tier architecture. At NewWays the development manager describes it in the following way: “Architecture is important to NewWays. Typically an application has three layers: At the bottom you have a database with con-

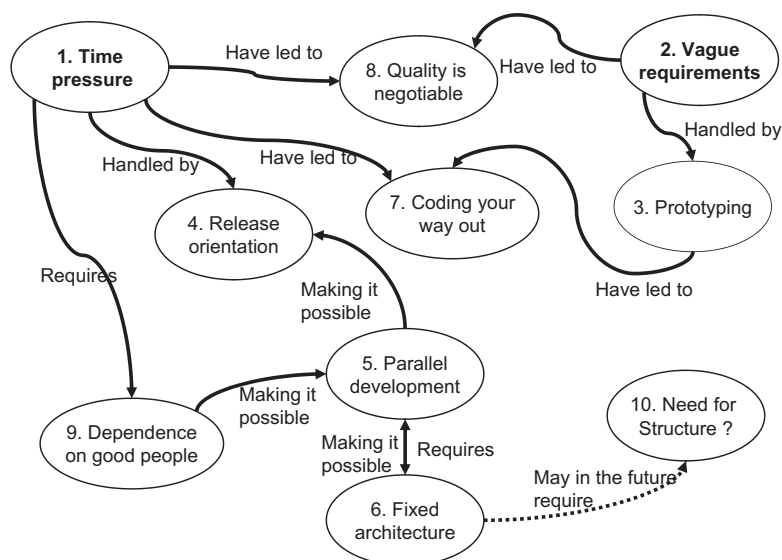


Fig. 1. Results from the first study.

tent; in the middle you have the business logic; and at the top you have the HTML generating logic, typically written in Visual Basic Scripts". The architecture is used as an important coordination mechanism to divide the work in the project. It is explained that: "Typically the graphical person is drawing something in PhotoShop which the HTML person then can cut up and put into tags", says one developer and another continues, "Which means that we are released from worrying about presentation and can concentrate on the heavy things" [i.e. the business logic and the database].

3.1.7. Coding your way out

The short time frame allowed for developing applications also introduces a coding focus or even hacking: "You have to accept that hacks are being made, that you don't have time to think systematically, and that you don't reuse because of the time pressure" (NewWays).

3.1.8. Quality is negotiable

Three different ways of looking and talking about quality have appeared over the last 20 years [27]. One school of thought focuses on fulfilment of customer expectations. Another way of thinking emphasizes measurable product attributes and conformance to requirements. The third approach is process oriented and assumes that a good development process will lead to quality. The three resulting kinds of quality can be named expectation-based, product-based, and process-based quality.

As a consequence of both time pressure and vague requirements we found that both product-based and process-based seemed to be ignored. Moreover, customers and users seemed to expect low quality. We decided to call this phenomena negotiable quality.

ProfWeb was for example struggling with quality. They knew it was not good enough and they had started thinking about what to do: "We collect a Test Group for every project. At least that is the plan for the future, but right now we are running the pumps, not financially, but we are very busy ... I have a capacity planning system and the UNIX department is booked 4 months ahead" (ProfWeb). Thus time pressure is a cause of the negotiable quality.

3.1.9. Dependence on good people

Time pressure is also the primary reason why good people are in high demand. As one of the founders of ProfWeb phrased it: "I believe the largest bottleneck we have is to get enough qualified employees". However, not all kinds of IT people were in high demand. Traditional analysts were not in as high demand as the technical people who were close to the code: "I also realised that the job market is such that I could find 25 new consultants tomorrow but I wouldn't be able to find two new programmers" (ProfWeb).

3.1.10. Need for new kinds of structure

An issue that is closely related to methodology and to a number of issues we have addressed above is structure. We have not been able to establish a solid causal relationship, but we have indications that seem to reveal that the older and larger the organization and/or the customers the larger the need for structure. For example, AlfaWeb, which only had existed for half a year when we interviewed them, was not feeling any need for structure. The CEO explained: "I believe it is the informality but also the lack of formal structures. If people have to close-knit a framework to work in they might cut down on creativity" (AlfaWeb). In contrast, NewWays, which had existed for 2 years and had 50 employees, had started creating some structures, and had started using a number of object-oriented techniques.

At this point in time, the low level of formal structures meant that the method components that constituted the way of working were continually ready for use; e.g. it did not take any preparation

time or cost to start the activity of 'coding your way out'. The time pressure and vague requirements were causing reactions to change as well as rapid creation of change in the form of prototypes and frequent releases. However, there was little time for improving the way of working according to lessons learnt. Moreover, scant attention was paid to ensuring the economic feasibility and quality of the developed software solutions, i.e. to creating value for the customer. Thus, the applied method components contributed to the practitioners' ability to react to change and to create software solutions fast, but they did not emphasize learning and customer value. We conclude that in 1999 High-speed development for the Internet showed early manifestations of agility (i.e. pre-agility) due to the focus on change and speed. However, the interviewees did not at this point in time use a particular formalized agile method; they did not refer to their way of developing as agile; and according to the method taxonomy [3] the development approach also did not yet support all four aspects of agility.

3.2. Study two results: A "new" software development process

The beginning of the millennium was characterized by a serious reappraisal of the concept and practicalities of ISD methods [28], as exemplified by discussions of amethodical development [29], formalized methods versus methods-in-action [30], and the advent of the agile manifesto. All these examples represented a critical stance towards the plan-driven approach to ISD. In 2001 we also found that the interviewed practitioners perceived their development practice to be radically different from 'traditional' development. Moreover, a key finding was that the case companies were getting good at developing software at high-speed by using an increasingly established set of practices that facilitate quick results. Fig. 2 shows that we identified three major categories of observations that were causing a (perceived) change to the software development process, and three major categories that were resulting from the changing causes [21].

3.2.1. A different kind of market environment

The Internet created a unique platform and marketplace for software products – one that was flexible in terms of requirements and quality. Requirements and quality were negotiable from release-to-release in a market-oriented process where competition and pragmatics were allowed to intervene to limit the scope of features in each release.

3.2.2. Lack of experience

The interviewees reported that there were too few knowledgeable and experienced developers who understood the new technology, changing market conditions, and who could meet the need for speed. A manager from Melpomene told us that "lots of people [in our organisation] came from more corporate environments where it took forever to get things out the door". Much of this prior experience was a hindrance rather than a benefit in the new environment. The shortage of experienced professionals made the marketplace for developers tight and expensive, and created development organizations that lacked sufficient experience and expertise.

3.2.3. Desperate rush to market

In all the case companies they explained that Internet software development was driven by a desperate rush to market. "Time-to-market ... Bigger, faster, better. Everything is very rush, rush, rush" (Polyhymnia).

3.2.4. Quality is negotiable

Many quality factors were not as critical in Internet speed development as they were in traditional software development.

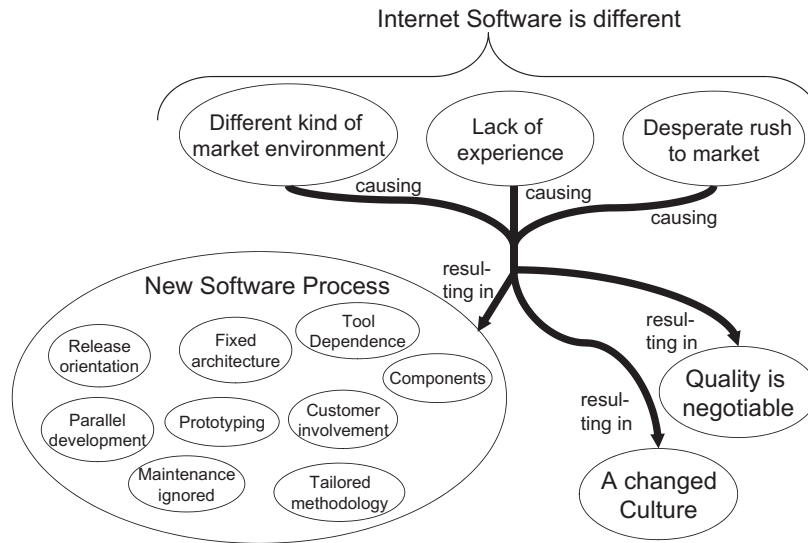


Fig. 2. Results from the second study.

Customers and users appreciated quick results and were willing to defer a certain amount of reliability and performance until later releases. And developers were willing to rebuild badly designed or coded features later when the deferment ran out. “It is different working at Internet speed. Compressed cycles mean that quality suffers. With speed we are sending poorer quality out the door” (Polyhymnia).

3.2.5. A changed culture

We found that Internet software development organizations had a distinct culture that appreciated informal structure, smaller teams, and diverse team compositions. Moreover, there seemed to be a tight bond among Internet software developers, a sense of belonging with others who shared the same values. “We are not 9 to 5 people down here. We are more dynamic ... There is a lot more excitement and enthusiasm here” (Thalia).

3.2.6. A “new” software process

At the project level, we identified a set of nine properties that characterized the “new” software development process (see Table 4). The collection of characteristics was aimed at producing quick results, and remarkably common in the case companies.

At this point in time, in 2001, the case companies were learning how to deal with the changed software development context, characterized by time pressure and a lack of both customer and developer understanding of business and technical opportunities and market developments. Thus, the case companies were getting increasingly good at applying a more and more established set of method components that facilitated quick responses to changing requirements and frequent delivery of scoped software solutions. Just as in the first study, the method components were continually ready for use, and supported a focus on change and speed. However, attention to customer value creation was still limited. Thus, also in this second study, no formalized agile methods were used. Instead the development approach is best characterized as pre-agility, emphasizing speed and change.

3.3. Study three results: Balancing speed and quality

Major changes took place from our second to our third study, i.e. in just 2 years [22]. At the market level two things had changed. *The changing market and IT economy* were having visible impact on the firms that we visited. The companies were still under pressure

to develop and deliver software at high-speed, but they were operating with significantly lower capital than before, and were no longer finding it difficult to hire or retain the talent they needed. Most of the companies described their states as either holding back or cutting down on their employees. Thus, changes in the IT economy were especially noticeable with respect to personnel and staffing. Moreover, disappearing venture capital as well as tight budgets forced the companies to focus on business value and costs. Thus, employment of a *set of business solutions* at the portfolio level were now carefully coordinated with use of a *set of technical solutions* at the project level.

Also contributing to the state of affairs at this point in time is the hard-won *experience* gained during both the dot com boom and bust. In our second study, the development staff at the companies we visited expressed boundless energy, excitement, as well as some confusion about what they were supposed to do and how. In our third study a more mature and reflective perspective was evident. One member at Melpomene suggests that caution contributed to the company’s survival: “At that time, there was excitement. But all the successful practices were successful because they were good practices. We made conservative decisions and are still here”. Thus, significant learning has taken place in these companies, across a wide range of business and technical topics. A member of Thalia sums up the learning in the following way: “Products are getting better due to more experienced developers”. Fig. 3 provides a visual overview of our results from the third study.

At the portfolio level, the changing market and IT economy resulted in increased attention to the need for business models and a new, or increased focus on costs. “What has changed? We don’t waste time on things that don’t generate revenue” (Thalia).

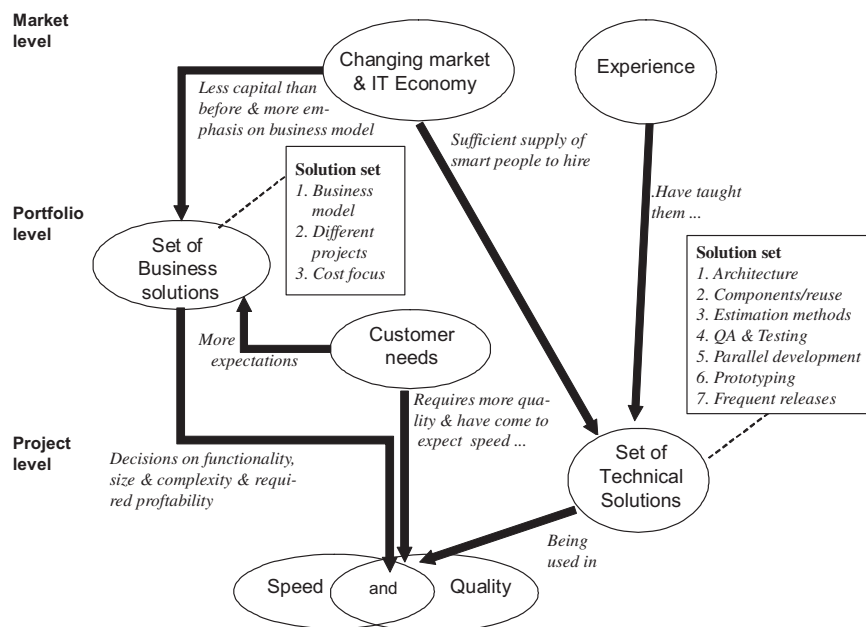
As opposed to the days of abundant resources where risky projects even with faint hopes of success were undertaken, organizations were now much less willing to fund projects that did not have a clear business case. “We have to balance the need to do things fast ... [with] a business case” (Urania). A manager at Thalia also explains that his “Product is expected to generate revenue. That is different than before. Now we need to make a business case for each project”. This situation encouraged project managers to clearly articulate the rationale for their projects, position them appropriately in alignment with organizational needs and requirements and in addition, market them to relevant decision makers.

All of the companies were refining their identities and offerings, but the challenges were being tackled in different ways. Some

Table 4

Nine characteristics of the new software process (study two).

Characteristic of the new software process	Description and examples
Parallel development	To achieve high-speed we found that companies compressed development into a time frame where only overlapping, parallel development could meet the demands
Release orientation	"People have a perception of Internet speed. They expect it. So we've had to scope our delivery or deliver a smaller set of features. Thereby releasing more often", said a manager from Euterpe. Clio said: "Development cycles last from 2 to 15 days... timing is important. Features that cannot be completed in time can slip from one release to the next. The fast cycle time softens the penalty from slipping a feature"
Tool dependence	Urania estimated that "fifty percent of development is already taken care of by tools we use such as iplanet or websphere. The APIs to these tools gives a lot of functionality". Many Internet software development organizations made heavy use of development tools and environments that could speed up the design and coding process. Furthermore new tools also helped to create well modularized and architected systems
Customer involvement	When requirements are fuzzy it helps having close access to customers. Thus intimately involving customers to cope with evolving and unstable requirements was typical. We also found that customers were often co-located with the development team, and participated closely in all phases of development. Most projects relied on such involvement rather than a formalized requirements management process
Prototyping	Instead of using formal requirements documents, most projects used prototyping as a way to communicate with their customers to validate and refine requirements. Customers would describe the basic functionality for new or changed features and these were quickly prototyped for demonstration and experimentation. "We are supposed to have a full [requirements and design document] but a lot of programmers use the prototype and go back and forth to check, or go back and ask: what was this supposed to do" (Melpomene)
Criticality of architecture	A well-planned architecture enable each release to be developed with some similarity. A three-layer architecture was common: (1) Database layer, (2) Business logic layer, the detailed processing code, and (3) User interface layer
Components based development and reuse	Internet speed can be achieved by software assembled with as many reusable components as possible, rather than crafted from scratch. "Internet speed needs reuse. We need to take components and to know how to put them together" (Thalia)
Maintenance ignored	The short life span of Internet software meant that maintenance often was not given serious consideration. "Products are not documented. No design document, no requirements specification. The person who did it is gone. It takes much longer time. Often we can start from scratch. It leads to a throw away mentality" (Polyhymnia)
Tailored methodology	The processes and methods used in Internet software development varied considerably depending on the composition of the project team and the nature of the product. "We have an overall methodology. But we have to tailor processes for individual teams" (Urania). Just "enough process to be effective", added Euterpe

**Fig. 3.** Results from the third study.

companies harkened back to more conventional business models, recognizing that "Success from now on depends on being a software and service company rather than an Internet company" (Melpomene), while others were forming partnerships with external (development) organizations.

3.3.1. Customer needs

The voice of the customer was still very much present, expressed through product strategy concerns, relationships, and on-going contact. "The speed hasn't changed. If anything it gets faster and faster as customer expectations grow... [the biggest challenge]

is meeting your customer's expectations" (Urania). Customer needs remained a challenge to discern and satisfy and at the same time customer expectations – for speed *and* quality – were significantly higher than in the second study.

At the project level, categories manifested themselves as a set of technical solutions. Of the seven categories in this solution set, five were somewhat similar to the process elements in study two. These five similar elements are: a standard architecture, the (re)-use of components, parallel development, prototyping, and frequent releases. Two new elements appeared: estimation and the improvement and involvement of quality assurance (QA) and testing. The two new elements are described in Table 5.

3.3.2. Speed and Quality

The need for speed was as constant in our third study as it was in the second study. "E-speed and e-haste are just normal now. Now you just know that you have to go that way and balance for quality" (Urania). Quality was associated with number of defects, customer satisfaction, and overall success. Our interviewees explained that "If you don't follow your processes, or do your documentation, that is not quality" (Urania) and that "They'll forget that you're late but they won't forget if it's bad" (Polyhymnia). Thus, at this point in time all three types of (product, process, and expectation-based) quality had become important.

Already in the second study the case companies had more or less adjusted to the time pressure that had come to characterize the E-economy. The time pressure resulted in use of method components that allowed for changing requirements and production of fast results. We referred to the almost exclusive focus on change and speed as pre-agility. In the third study not only time pressure but also economy and quality had become important; the latter due to the dot com demise. To support the new emphasis on value adding software solutions, method components concerned with development of a business case, estimation, and QA and testing had been introduced. It seemed that the case companies were learning what it takes to release valuable, working software often, namely prioritization and estimation of projects and functionality as well as (automatic) testing skills. Thus, at this point in time the case companies were employing a set of method components that were continually ready to use and which facilitated a focus on change, speed, and value creation. We therefore characterize the development approach as supporting agility. However, the practitioners did not yet conceptualize the applied practices as agile development, and they also did not apply dedicated agile methods.

3.4. Study four results: Boundary spanning with Scrum

Five years later, in 2008, agile development had become a known term. In the fourth study, we examined three companies that had implemented the agile method Scrum. Common for all three companies was that they were applying Scrum for some parts of their software development process and a more traditional

approach for other parts of their development efforts. The case companies were motivated to use Scrum by an internal drive to achieve the benefits of an agile approach. The following benefits were highlighted as particularly important:

1. A closer contact with and immediate feedback from the customer.
2. Increased developer commitment and feelings of ownership.
3. The energy released from being able to focus on quick results.

At the same time, and due to the size and distributed nature of the team work as well as the criticality of the software, the interviewees stressed the need for alignment. Alignment is described as necessary to ensure that the work carried out by the Scrum teams is in line with the overall scope document, budget, and project plan. The work should also align with the major milestones of the project and broader company-prescribed methods and standards (e.g. CMMi). The wish for energy and agility and the need for overview and alignment has led all three case companies to employ 'a mixed strategy' [6] where they combine the relatively recently adopted agile approach with more well-established plan-driven ways of working (see Fig. 4).

The three companies are very clear about how they organize to achieve the benefits of both the agile and the plan-driven approach. They explain that "We carry out project planning and management at several levels". (SuperSystem). Thus, in all three cases, the Scrum team(s) and master(s) are allowed to have a narrow focus on 'today, tomorrow, this sprint, and the next', while a project manager has the responsibility for the project and the overall alignment of plans and people, i.e. for the big, and long-term picture. This division of work is necessary because "Scrum does not help with the overall, long-term planning of the project...you need to have an additional layer of project management. Otherwise it is not possible to coordinate and oversee a project with a deadline 1½–2 years into the future" (SuperSystem).

As the companies have chosen to organize for both agility and alignment certain Scrum elements and key people come to play a boundary spanning role that facilitates the sharing and negotiation of knowledge between several intersecting, but distinct social worlds [31,32].

3.4.1. Agility

The companies explain that the development teams are "...true to Scrum" (SuperSystem) and that they "...use Scrum more or less 'by the book'" (GlobeRiver). Thus, Scrum helps the Scrum team conduct the work that is a part of their social world, namely the coordination and performance of tasks (related to analysis, design, development, test, integration, and release) as well as the monitoring of progress, risk, and quality for the current sprint. Moreover, Scrum plays an important role in providing a number of boundary objects that mediate the interaction that takes place between the Scrum team and the customer organization. For the product owner

Table 5
Two distinctive characteristics of the technical solution set (third study).

Characteristics of the technical solution set	Description and examples
Estimation	A major difference between our second and third study was the recognition of the need for good estimation methods to track and improve performance. The organizations that had been involved in internet software development for a few years declared that they were more mature in their estimation of effort and schedule – "we know what it is like to develop in this environment" (Polyhymnia)
Improve and involve QA and testing	With markets and products maturing, quality was getting more important. QA and testing was now seen as important aspects of software development. Due to the time-constrained environment, the need for more efficient QA was stressed. "If I look at a project time line, a lot of it is in QA testing. We need to improve and automate and create scripts to drive that down" (Thalia)

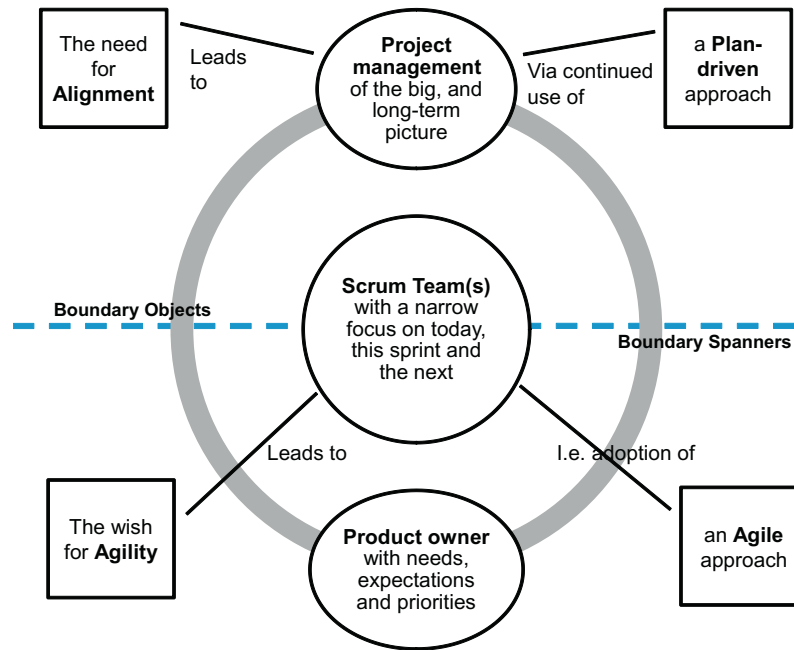


Fig. 4. Results from the fourth study.

the prioritized user stories constitute functionality that will be a part of the next deliverable, and for the developers the user stories are (also) tasks that have to be carried out during the particular sprint. At the same time, the stories and the software allow the team members and the product owner to communicate, share knowledge, and create new meaning across the boundaries of their different worlds.

3.4.2. Alignment

In all three case companies software development was previously conducted in accordance with a sequential, document-oriented development model, and a plan-driven approach continues to be used after the introduction of Scrum. However, the plan-driven approach is now separated from the development teams and activities, and used by an appointed project manager for overseeing the project as a whole. It is explained that "...surrounding the team's work and the burn-down chart is the overall project plan, including milestones, broad-level estimates, a mapping from milestones to sprints, and a plan over external releases, as well as a risk analysis. The board-level estimates and the mapping from milestones to sprints help validate if the project and its scope can be achieved within the time frame and the budget, but the details of the sprints are not specified in these plans. That is the responsibility of the team" (SuperSystem).

DareYou also reported that, even though they are the customer organization, they consider themselves responsible for the project and its success. Consequently, they also operate with an appointed 'traditional' project manager, as well as project management tools and documents such as a written project vision, budget, overall project plan and some up-front specification of requirements. In this case, the customer's project manager is the boundary spanner who keeps the project and its plans and participants aligned, and together with the product owner she is heavily involved in and well-informed about the actual quality and progress of the suppliers' development efforts.

In GlobeRiver the overall management and responsibility for the development of new engineering products resides with the R&D department. Thus, the Indian Scrum teams carry out the software development within the frame of large, business critical pro-

jects that involve many internal departments and external suppliers and which are managed in accordance with a traditional plan-driven approach. It is very important that the software meets the deadlines in the road map for the product development project as a whole and that it is in line with the requirements and quality in the specification. In this setting, the R&D project manager is also the product owner and responsible for prioritizing the already specified functionality, which the Scrum teams then develops during a number of sprints. Moreover, a third role, a Facilitator, has been introduced. The Facilitator, located in Denmark, "...is the main point-of-contact between the Danish product owner and the Indian teams and follows-up on progress and impediments on a weekly basis, or more if needed. ..." (GlobeRiver). The Facilitators serve as boundary spanners who use certain information objects (such as, e.g. the road map, product backlog, burn-down chart, and impediments list) to keep the Indian Scrum team informed about requirements, priorities, and deadlines and the Danish R&D manager up to date about progress, quality, and risks. In this way, the Facilitator plays an important role for the translation of information between the agile and the plan-driven worlds. This in turn allows the Indian software developers and the Danish R&D personnel to operate almost completely according to their own goals and work practices.

In all three case companies, the combined use of Scrum and a plan-driven approach has been organized so that the involved agile and plan-driven communities-of-practice [33] can work largely in keeping with their own goals, information needs, and methods. Consequently, the translation of information and negotiation of new meaning across the different intersecting worlds is necessary. To this end, certain information elements (i.e. the overall project plans and burn-down charts) function as boundary objects, while the Scrum masters in SuperSystem, the Project manager in DareYou, and the Facilitator in GlobeRiver have boundary spanning roles, which they are fully aware of.

In the third study, the applied method components supported agility. Also in the fourth study the case companies' development approach is characterized by agility, as the Scrum teams use method components that are continually ready to use and emphasize change, speed, and value creation. For example, Scrum provides

method components that facilitate reactions to and prioritization of changing requirements, creation of change via frequent releases of valuable, working software, and learning from change in regular retrospectives. In contrast, the intention of the plan-driven project management layer is not to support change. However, it is the project manager's (or Facilitator's) task to ensure the alignment of overall plans, people, and on-going tasks, and therefore changes do take place. We conclude that while the applied plan-driven method components do not in themselves aim to facilitate creation of, reaction to or learning from change, their use also do not detract from the development teams' ability to achieve agility.

Importantly, at this point in time agile development has become a distinct phenomenon: something that is 'implemented' in the form of a particular agile method, which is then subsequently 'followed by the book' and which is operating alongside other ways of working. Thus, in 2008, the interviewed practitioners conceptualized their practice as agile development; they clearly defined and delimited the boundaries within which agile development was in use; and they explained how certain method components and key people were able to span the boundaries of the agile and plan-driven worlds.

4. Discussion and conclusion

Over the 10-year time span, learning has been generated from each of the four studies. Much of this learning might be characterized as detailed and "keen insight" that is created within each of the studies and which does not bear well in brief summaries. However, it is possible to consider the learning that arises from the accumulation of insights across the four studies. A limitation of this approach lies in its assumption that the four studies provide serial episodes in a continuing saga. Because the studies involve differing subject organizations and individuals, we cannot rule out the alternative explanation that the consistencies in the data sets are accidental.

With this caveat in mind, we can summarize and interpret the collection of four studies as follows. The central story line in the

first study brought the changing landscape of software development into sharp focus. In this study, it appeared that the two main sources driving software development were incredible time pressures coupled with unknown and changing requirements. These two primary causal factors arose as the context for software development changed due to the emergence of the E-economy.

The central story line in the second study embraced a new software process that was common across the respondents. This process included customer involvement, a release orientation, etc. The components of the new process were present in the first study, but had become more or less established practice in the second study [34].

In the third study, the story shifted again, but this time away from software development in a local sense. This story line focuses instead on changing economics and the role of software in formulating business solutions and generating revenue. A balancing game arises in which business and technical factors are brought together by high-speed software projects.

Finally, in the fourth study, the central story line shifts once more, but the focus falls back on software process. In this study, we find organizations which are not exactly integrating agile and planned software processes; rather they are operating these two different ways of working consistently within separate boundaries. Work is flowing across the boundaries to enable the organizations to harvest the benefits they require from each of the deployed software processes (agile and planned). Thus, boundary objects and spanners play a key role in this story.

An interesting aspect of these study settings is the historically repeating two-stage pattern where the story line first centers on a changing context and then on the software process, and where somewhat chaotic, context-driven process adaptations seem to be followed by a more stable period of software process usage (see Fig. 5).

Thus, our studies reveal a two-stage pattern in which dramatic changes in the market causes disruption of established practices, experimentation, and process adaptations followed by consolidation of lessons learnt into a new and once again relatively stable

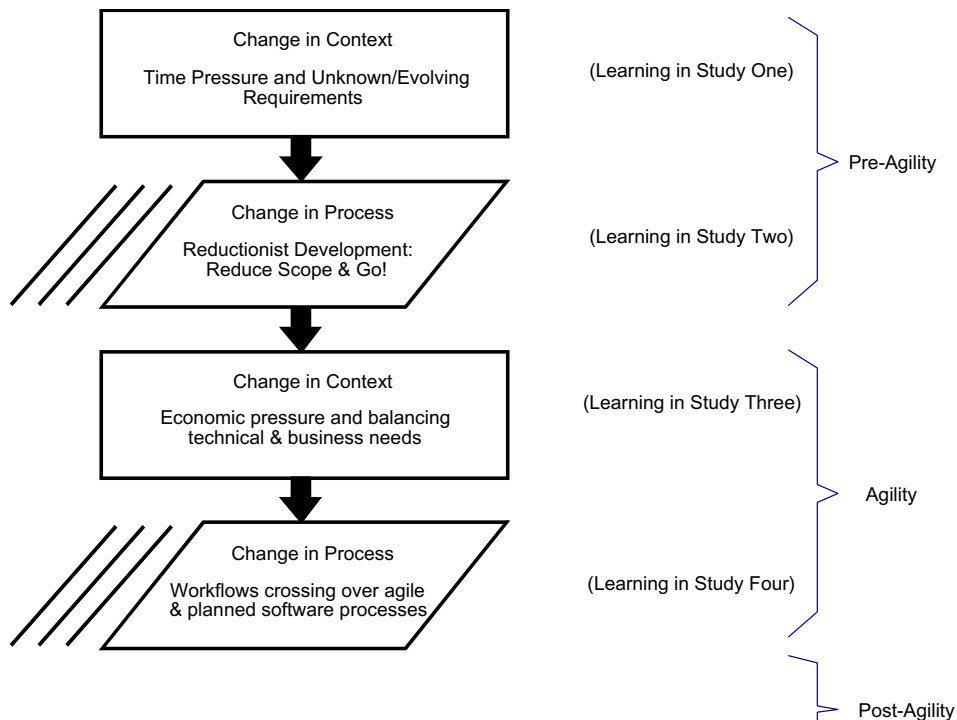


Fig. 5. Learning across the four studies.

software development process. This suggests that reoccurring periods of radically changing context will interrupt software process evolution and that progression in software processes may therefore occur in punctuated historical cycles. Software process evolution does not necessarily restart from ground zero in each new episode. Our evidence clearly indicates that our settings learn from and build on previous experience. However, the evidence does suggest that each episode begins with some confusion and reversal before it advances. This pattern is similar to a cycle of exploration and innovation followed by asset exploitation noticeable in some organizational learning strategies [35].

Our understanding of this punctuated evolution helps to historically situate the constitution of agile ISD. Our two first studies predated the agile manifesto. Moreover, early phenomena such as “Internet Speed” and “Internet Time” were not “agile development” as currently defined. Therefore, we can now reflect on these phenomena as pre-agility, an early form of agility that does not completely satisfy today’s taxonomies, but instead helped to shape them.

We can appreciate this formative pre-agility using Conboy’s [3] taxonomy of ISD agility. The taxonomy includes three basic categories. Two of these categories regard (1) continual availability of method components and (2) rapid change. These two aspects of agility are plainly present in study one and study two. However, the third category (which deals with perceived economy, quality, and simplicity; i.e. with the business case) is not entirely present. Quality in particular is considered negotiable in these early forms. In studies three and four, this had changed, and value creation had become a prominent part of the development approach.

An important aspect of this progression of agile ISD as a practical phenomenon is the repeating effect of the software process evolution itself in shaping the changing context. The context characterized by frantic centralization of rapid development and first-to-market development led to processes that defocused quality and cost. The resulting problems became part of the context. In other words, the reductionist development that dominated study two was both part of the solution to the problems in the context discovered in study one, and part of the problem in the context discovered in study three. The evolution of software development processes both defines the solution to current dominant problems, and shapes the problems for the next cycle.

This cyclical evolution has interesting implications for the future of agile development. Our findings from the forth study indicate that the software development process has once again reached a state of some stability. We may therefore in study four be able to recognize the seeds of the next episode of changing context. These seeds may surface as key problems that shape the context of the next episode. For example, this could be the organizational issues created by the boundaries between agile development teams and plan-driven personnel. The organizations in study four must work to coordinate the workflow on each side of the boundary, and the workflows that transit across the boundary. If such a boundary sets the limit on the usefulness of and application area for agility, then the next cycle of this evolution might seek to eliminate the boundary. We can only speculate on how this might be accomplished, but it seems likely that it could be via software development approaches that more seamlessly integrate plan-driven and agile features. The result could be a form of development that we might regard as “post-agility”. Forms of post-agile ISD would not distinguish between agile and plan-driven development and methods or be concerned with creating and spanning organizational and workflow boundaries. *Instead the focus would shift towards proactively pursuing the dual goal of agility (i.e. fast responses to changing requirements, and frequent releases of valuable, working software) and alignment (of plans, people, and tasks) through a diversity of means*, for example through (agile, plan-driven, and other types

of) method components, and software tools as well as via new ways of organizing, specializing, communicating, managing relationships, etc. Thus, by “post-agility” we do not imply the end of agility; rather it refers to a deep incorporation of agility into all modes of software development such that agile and plan-driven cease to be distinguishable. In other words, post-agility is also a post-plan-driven mode of development.

Our empirical findings and speculations about the future have a number of implications for research. First, the learning that we draw from across the four studies indicates that it would be constructive to broaden the primarily local, intentional, and model driven way we view and research software process evolution. Second, our findings show that a boundary spanning perspective is a useful theoretical lens for understanding the current success and stability of agile development. Third, at the same time our findings about the two-stage pattern and the proposed notion of post-agility indicate that other theories might be needed to understand and provide support for the next stage of agile ISD process evolution.

Finally, there are interesting implications from the longevity of this study. A series of empirical studies that span a decade is unusual. There are at least four reasons why the studies have continued. First, the initial study was an exploratory examination of the genesis of a particular phenomenon: agile systems development. Indeed, the empirical work on this study occurred before the approach was so named or defined. The researchers recognized that poorly understood practices were congealing across many organizations at the same time. Second, there was considerable enthusiasm and advocacy for these agile processes among those engaging in the practices. It helped that the processes were viewed as controversial and revolutionary by some. The researchers were warmly welcomed by practice (in some cases, again and again) making for easy access to empirical data. Third, the phenomenon of agile development has continued to evolve over the 10-year time period. Fourth, the cyclical history helps us distinguish pre-agility from current practices (agility), and based on the new problem space, to speculate about post-agility, the possible next cycle in this history. Fifth, and finally, the basic research method applied at the four study periods has proved to be durable across time. The grounded theory that emerged from each of the studies has been unique. However, these independent core story lines have collectively materialized into a consistently repeating pattern. Thus, through analysis of the individual story lines and their common themes, a global theory, still anchored to the empirical data, has emerged across our studies.

References

- [1] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: a systematic review, *Information & Software Technology* 50 (2008) 833–859.
- [2] P.J. Ågerfalk, B. Fitzgerald, S. Slaughter, State of the art and research challenges, *Information Systems Research* 20 (2009) 317–318.
- [3] K. Conboy, Agility from first principles: reconstructing the concept of agility in information systems development, *Information Systems Research* 20 (2009) 329–354.
- [4] R.D. Austin, L. Devin, Research commentary—weighing the benefits and costs of flexibility in making software: toward a contingency theory of the determinants of development process design, *Information Systems Research* 20 (2009) 462–477.
- [5] A.H. Van de Ven, *Engaged Scholarship: A Guide for Organizational and Social Research*, Oxford University Press, New York, 2007.
- [6] P. Abrahamsson, K. Conboy, W. Xiaofeng, Lots done, more to do: the current state of agile systems development research, *European Journal of Information Systems* 18 (2009) 281–284.
- [7] R. Baskerville, J. Pries-Heje, S. Madsen, From exotic to mainstream: a 10-year odyssey from Internet speed to boundary spanning with Scrum, in: T. Dingsøyr, T. Dybå, N.B. Moe (Eds.), *Agile Software Development: Current Research and Future Directions*, Springer, Heidelberg, 2010, pp. 87–110.
- [8] A. Strauss, J. Corbin, *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*, second ed., Sage, Thousand Oaks, CA, 1998.
- [9] W. Orlikowski, CASE tools as organizational change: investigating incremental and radical changes in systems development, *MIS Quarterly* 17 (1993) 309–340.

- [10] C. Urquhart, Exploring analyst–client communication: using grounded theory techniques to investigate interaction in informal requirements gathering, in: A.S. Lee, J. Liebenau, J.I. DeGross (Eds.), *Information Systems and Qualitative Research*, Chapman and Hall, London, 1997, pp. 149–181.
- [11] C. Urquhart, Strategies for conversation and systems analysis in requirements gathering: a qualitative view of analyst–client communication, *The Qualitative Report* (On-line Serial) 4 (2000).
- [12] R. Baskerville, J. Pries-Heje, Racing the e-bomb: how the Internet is redefining information systems development methodology, in: N.L. Russo, B. Fitzgerald, J.I. DeGross (Eds.), *Realigning Research and Practice in Information Systems Development: The Social and Organizational Perspective*, Kluwer Academic Publishers, Norwell, Massachusetts, USA, 2001, pp. 49–68.
- [13] L. Rising, N.S. Janoff, The Scrum software development process for small teams, *IEEE Software* (2000) 26–32.
- [14] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2000.
- [15] K. Beck, M. Fowler, *Planning Extreme Programming*, Addison-Wesley, Boston, 2001.
- [16] R. Jeffries, A. Anderson, C. Hendrickson, *Extreme Programming Installed*, Addison-Wesley, Boston, 2001.
- [17] B. Boehm, Get ready for agile methods, with care, *IEEE Computer* 35 (2002) 64–69.
- [18] B. Boehm, R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*, Addison-Wesley, Boston, 2004.
- [19] A. Cockburn, Learning from agile software development – Part one and two, *Crosstalk – The Journal of Defence Software Engineering* (2002).
- [20] M. Cusumano, D. Yoffie, Competing on Internet Time: Lessons from Netscape and Its Battle with Microsoft, Touchstone, New York, 2000.
- [21] R. Baskerville, L. Levine, J. Pries-Heje, B. Ramesh, S. Slaughter, Is Internet-speed software development different?, *IEEE Software* 20 (2003) 70–77.
- [22] J. Pries-Heje, R. Baskerville, L. Levine, B. Ramesh, The high speed balancing game: how software companies cope with Internet speed, *Scandinavian Journal of Information Systems* 16 (2005) 11–54.
- [23] R. Baskerville, J. Pries-Heje, Information systems development @ Internet speed: a new paradigm in the making, in: S. Wrycza (Ed.), *Proceedings of the Tenth European Conference on Information Systems*, University of Gdansk, Gdansk, 2002, pp. 282–291.
- [24] P.G. Smith, D.G. Reinertsen, *Developing Products in Half the Time*, second ed., Van Nostrand Reinhold, New York, 1995.
- [25] M. Cusumano, R. Selby, *Microsoft Secrets: How the World's Most Powerful Company Creates Technology, Shapes Markets and Manages People*, Free Press, New York, 1995.
- [26] M. Iansiti, A. McCormack, Developing products on Internet time, *Harvard Business Review* 75 (1997) 108–117.
- [27] P.B. Crosby, *Quality is Free: The Art of Making Quality Certain*, New American Library, New York, 1980.
- [28] D. Avison, G. Fitzgerald, Methodologies for developing information systems: a historical perspective, in: D. Avison, S. Elliot, J. Krogstie, J. Pries-Heje (Eds.), *The Past and Future of Information Systems: 1976–2006 and Beyond*, Springer, Heidelberg, 2006, pp. 27–38.
- [29] D.P. Truex, R. Baskerville, J. Travis, Amethodical systems development: the deferred meaning of systems development methods, *Accounting Management and Information Technologies* 10 (2000) 53–79.
- [30] B. Fitzgerald, N.L. Russo, E. Stolterman, *Information Systems Development: Methods in Action*, McGraw-Hill, London, 2002.
- [31] S.L. Star, J.R. Griesemer, Institutional ecology, 'Translations' and boundary objects: amateurs and professionals in Berkeley's Museum of Vertebrate Zoology, 1907–39, *Social Studies of Science* 19 (1989) 387–420.
- [32] N. Levina, E. Vaast, The emergence of boundary spanning competence in practice. Implications for implementation and use of information systems, *MIS Quarterly* 29 (2005) 335–363.
- [33] A. Cox, What are communities of practice? A comparative review of four seminal works, *Journal of Information Science* 31 (2005) 527.
- [34] R. Baskerville, J. Pries-Heje, Short cycle time systems development, *Information Systems Journal* 14 (2004) 237–264.
- [35] J.G. March, Exploration and exploitation in organizational learning, *Organization Science* 2 (1991) 71–87.